



American International University-Bangladesh (AIUB)

Department of Computer Science

Course Name: **COMPUTER GRAPHICS**

Faculty Name: DIPTA JUSTIN GOMES

Section: J

Student Name: **NILOY PAUL**

Course Code: CSE 4118

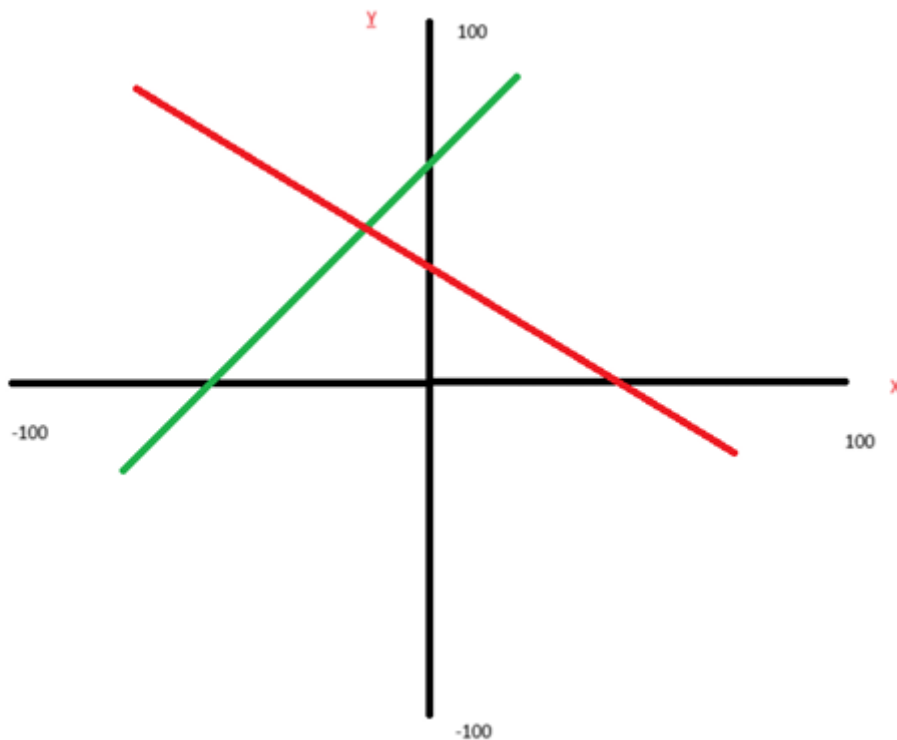
Semester: **FALL 2025-2026**

MID Term LAB Task-3

Student ID: **23-51773-2**

DDA and Integer Pixel Grid

1. Construct a OpenGL Program containing the following conditions:
 - a. Use a separate init() function to declare your own co-ordinate system. Use the range (-100, 100, -100, 100).
 - b. Construct a display function using DDA Algorithm to generate a straight line with input: X1, Y1 (initial vertex) and Xn,Yn (Final Vertex). Consider all three conditions of the DDA Algorithm while implementing the algorithm.
 - c. Create two lines as given below using DDA algorithm:



OpenGL Code

```
#include<windows.h>
#include<iostream>
#include<math.h>
#include <stdio.h>
#include<GL/gl.h>
#include <GL/glut.h>

using namespace std;

int X1, Y1, X2, Y2, X3, Y3, X4, Y4;

void MidPointsLINE(int x0, int y0, int xn, int yn)
{
    int dx = xn - x0;
    int dy = yn - y0;
    int x = x0, y = y0;

    // Color: green if dy/dx >= 0, red otherwise
    if ((float)dy / dx >= 0)
        glColor3ub(0, 255, 0);
    else
        glColor3ub(255, 0, 0);

    glBegin(GL_POINTS);

    if (abs(dx) >= abs(dy))
    { // |m| <= 1 → drive along x
        int d = 2*abs(dy) - abs(dx);
        int dE = 2*abs(dy);
        int dNE = 2*(abs(dy) - abs(dx));
        int sx = (dx > 0) ? 1 : -1;
        int sy = (dy > 0) ? 1 : -1;
```

```

while (x != xn)
{
    glVertex2i(x, y);
    if (d < 0)
        d += dE;
    else
        { d += dNE; y += sy; }
    x += sx;
}
}

else
{ // |m| > 1 → drive along y
    int d = 2*abs(dx) - abs(dy);
    int dE = 2*abs(dx);
    int dNE = 2*(abs(dx) - abs(dy));
    int sx = (dx > 0) ? 1 : -1;
    int sy = (dy > 0) ? 1 : -1;

    while (y != yn)
    {
        glVertex2i(x, y);
        if
            (d < 0) d += dE;
        else
            { d += dNE; x += sx; }
        y += sy;
    }
}

glVertex2i(xn, yn); // plot endpoint
glEnd();
glFlush();
}

```

```

void MidPoints()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0,0,0);
    glBegin(GL_LINES);
        glVertex2i(-100,0); glVertex2i(100,0);
        glVertex2i(0,-100); glVertex2i(0,100);
    glEnd();

    MidPointsLINE(X1, Y1, X2, Y2);
    MidPointsLINE(X3, Y3, X4, Y4);
    glFlush();
}

```

```

void myInit(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glPointSize(3.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-100, 100, -100, 100);
}

```

```

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(150, 150);
    glutCreateWindow("Midpoint Line Algorithm - CG Sec J");

    cout << "Enter initial point for line 1 [X1 Y1]: "; cin >> X1 >> Y1;
    cout << "Enter final point for line 1 [X2 Y2]: "; cin >> X2 >> Y2;
}

```

```
cout << "Enter initial point for line 2 [X3 Y3]: "; cin >> X3 >> Y3;
cout << "Enter final point for line 2 [X4 Y4]: "; cin >> X4 >> Y4;

myInit();
glutDisplayFunc(MidPoints);
glutMainLoop();
}
```

Output



THE END